

공식 문서 기반 Skill / MCP 리뷰

대상: `reverse-spec/SKILL.md` , `reverse-prd/SKILL.md` 기준 문서(공식): `code.claude.com/docs/en/skills.md` , `.../commands.md` , `.../mcp.md` 작성일: 2026-06-23 · **최신화:** **2026-07-02** (8장 "현행 구조 스냅샷"에 이후 변경 반영)

/ 검토 범위: SKILL.md 프론트매터 스펙 준수 여부, 디렉토리 구조, `allowed-tools` 포맷, MCP 도구 사용/표기 규칙. 공식 프론트매터 필드 표를 기준으로 사용했다.

1. 요약 (Verdict)

항목	판정
프론트매터 필드 유효성	통과 — 사용된 필드(<code>name</code> , <code>description</code> , <code>argument-hint</code> , <code>allowed-tools</code>) 모두 공식 스펙에 존재
잘못된/비공식 필드	없음 — <code>allowedTools</code> (camelCase), <code>license</code> , <code>metadata</code> 등 비공식 필드 미사용
디렉토리 구조	통과 — <code>skill-name/SKILL.md</code> 규칙 준수
<code>allowed-tools</code> 포맷	통과 — 콤마 구분 문자열 + <code>Bash(pattern)</code> 는 공식 지원 포맷
<code>\$ARGUMENTS</code> 사용	통과 — 공식 치환 변수
MCP 도구 표기	해당 없음 — MCP 도구를 쓰지 않으며, 현 기능상 불필요 (아래 5장 참조)

결론: 두 스킬은 공식 SKILL.md 스펙을 준수한다. 차단성 결함(blocking)은 없고, 아래는 개선 권고(권장/정보성) 항목이다.

2. 필드별 검증 (공식 프론트매터 표 대조)

공식 문서: "All fields are optional. Only `description` is recommended so Claude knows when to use the skill."

필드	현재 값	공식 스펙	판정
<code>name</code>	<code>reverse-spec</code> / <code>reverse-prd</code>	선택. 디렉토리 기반 스킬에서는 커맨드명이 디렉토리명에서 결정되고, <code>name</code> 은 plugin-root SKILL.md에서만 권위를 가짐	동작상 무시됨(장식용). 디렉토리명과 일치하므로 무해
<code>description</code>	트리거 문구 포함 한국어 설명	권장 필드. 평문, listing에서 1,536자에서 잘림. 핵심 use case를 앞에	양호 (분량 충분히 짧음, 트리거 포함)
<code>argument-hint</code>	<파일_또는_디렉토리_경로> <code>[--format ...]</code>	선택. 자동완성 시 표시되는 평문 힌트	유효
<code>allowed-tools</code>	(최초 리뷰 시) Bash(<code>python *</code>) 등 광범위 → (현행) 스크립트 경로 한정 패턴, 8장 참조	선택. 공백/coma 구분 문자열 또는 YAML 리스트. Bash(<code>git add *</code>) 형태 패턴 허용	포맷 유효 (보안 권고 R-2 적용 완료)

YAML folded scalar(`>`)로 작성된 `description` / `allowed-tools` 는 단일 문자열로 접하며, coma 구분 문자열은 공식이 허용하는 포맷이라 문제없다.

3. 디렉토리 / 활성화 위치

- 현재: `reverse-spec/SKILL.md` , `reverse-prd/SKILL.md` (저장소 루트)
- 공식 프로젝트 스킬 경로: `.claude/skills/<dir>/SKILL.md` , 개인 스킬 경로: `~/.claude/skills/<dir>/SKILL.md`

권고 (R-1) — 적용됨: 이 저장소는 스킬을 배포/수집하는 컬렉션 성격이라 루트 배치는 합리적이다. 다만 이 스킬을 실제로 활성화하려면 `.claude/skills/` 아래에 있어야 자동 로드된다. README에 설치 경로(예: `cp -r reverse-prd ~/.claude/skills/`)를 명시하거나, 프로젝트 내 활성화가 목적이면 `.claude/skills/` 로 이동할 것.

4. `allowed-tools` 보안 관점

공식 정의: "Tools Claude can use without per-use approval while skill is active." 즉 `allowed-tools` 에 올린 도구는 스킬 활성 동안 **개별 승인 프롬프트 없이** 실행된다.

- `Bash(pip install *)`, `Bash(python *)` 는 사실상 **임의 코드 실행을 사전 승인**하는 셈이다.
- PDF/docx 생성을 위해 기능상 필요하긴 하나, 광범위한 권한이다.

권고 (R-2) — 적용됨: - 의도된 동작임을 SKILL.md 또는 README에 명시 (사용자가 권한 범위를 인지하도록). - 가능하면 범위를 좁힌다. 예: 생성 스크립트를 `scripts/` 로 분리하고 `Bash(python */scripts/render.py *)` 형태로 한정 (아래 R-3 참조). **주의:** `${CLAUDE_SKILL_DIR}` 치환은 스킬 **본문에 서만** 동작하며 프론트매터 `allowed-tools`에서는 확장되지 않는다(리터럴 문자열로 취급되어 어떤 명령 과도 매칭 안 됨). 따라서 허가 패턴에는 경로 와일드카드를 사용해야 한다. - `pip install` 은 사전 설치를 전제하면 제거 가능 (런타임 설치 회피).

5. MCP 리뷰

현황: 두 스킬 모두 MCP 도구를 참조하지 않는다. 그리고 현 기능 범위에서는 그게 맞다. 역기획 문서 생성은 로컬 파일 읽기 + 로컬 PDF/docx 생성으로 완결되므로 외부 MCP 서버가 불필요하다.

향후 산출물을 외부 시스템에 게시(예: Confluence/Notion/Jira/Slack)하도록 확장한다면, 공식 MCP 도구 표기 규칙을 따라야 한다:

- 표준 MCP 서버: `mcp__<server-name>__<tool-name>` (예: `mcp__github__create_pr`)
- 플러그인 번들 MCP 서버: `mcp__plugin__<plugin-name>__<server-name>__<tool-name>` (A-Z a-z 0-9 _ - 외 문자는 _ 로 치환)

이때 해당 도구를 `allowed-tools` 에 풀네임으로 추가한다. 예:

```
allowed-tools: >
  Read, Glob, Bash(python *),
  mcp__notion__create_page, mcp__confluence__create_content
```

권고 (R-4): 지금은 MCP 미사용이 적절. 외부 게시 기능을 넣을 때만 위 표기를 도입.

6. 구조 개선 권고 (선택)

권고 (R-3) — 적용됨: (이전) PDF/docx 생성 Python 코드가 SKILL.md 본문에 인라인으로 있었다. 공식 문서는 실행 스크립트를 `scripts/` 에 두고 `${CLAUDE_SKILL_DIR}` 로 참조하는 "supporting files" 구조를 권장한다. 분리 시 이점:

- SKILL.md가 가벼워져 모델이 지침을 더 잘 따른다.
- `allowed-tools` 를 `Bash(python */scripts/render.py *)` 로 좁혀 R-2 보안 권고도 충족. (프론트매터는 `${CLAUDE_SKILL_DIR}` 를 확장하지 않으므로 경로 와일드카드 패턴 사용.)

현행 구조 (적용 결과):

```
reverse-prd/                                # reverse-spec도 동일 구조
├─ SKILL.md
├─ reference.md                             # 1-A~1-H 공통 파싱 규칙 (reverse-spec과 동일 사본)
└─ scripts/
    ├─ extract.py                          # 결정적 사실 추출기 (1-A~1-H 구현, --emit-flow)
    ├─ flowgen.py                          # 유저플로우 SVG 생성기 (HTML·PDF 겸용)
    └─ render.py                           # Markdown → PDF+HTML(기본 쌍) / DOCX 렌더러
```

권고 (R-5) — 적용됨: (이전) `reverse-spec` 과 `reverse-prd` 의 Step 1(코드 파싱) 로직이 중복이었다. 공통 `reference.md` 로 분리하고 양쪽 SKILL.md에서 참조하면 유지보수성이 오른다.

7. 액션 아이템 정리

ID	구분	내용	우선 순위
R-1	적용됨	README에 설치 경로(<code>~/.claude/skills/</code> · <code>.claude/skills/</code>)·사용법·구조 명시	중
R-2	적용됨	<code>allowed-tools</code> 를 <code>Bash(python */scripts/render.py *)</code> 로 축소, <code>pip install</code> 도 패키지 한정	중
R-3	적용됨	인라인 Python을 <code>scripts/render.py</code> 로 분리, <code>\${CLAUDE_SKILL_DIR}</code> 참조로 호출	중
R-4	정보	MCP는 외부 게시 확장 시에만 <code>mcp__server__tool</code> 표기로 도입	하

ID	구분	내용	우선 순위
R-5	적용됨	Step 1 공통 파싱 로직을 <code>reference.md</code> 로 분리, 양쪽 SKILL.md에서 <code>\${CLAUDE_SKILL_DIR}/reference.md</code> 참조	하
—	정보	<code>name</code> 필드는 장식용(무해). 제거해도 됨	하

적용 이력 (2026-06-26): R-2·R-3·R-5는 리팩터링으로 반영됨 — 두 스킬이 `reference.md` (공통 파싱)와 `scripts/render.py` (공통 렌더러)를 공유하고, `allowed-tools` 는 스크립트 경로로 한정됨. R-1은 저장소 `README.md` 로 충족. R-4(MCP)는 현 기능상 불필요하여 정보성으로 유지.

정정 (2026-07-02): 최초 R-2 적용 시 사용한 `Bash(python ${CLAUDE_SKILL_DIR}/scripts/*)` 패턴은 프론트매터에서 변수가 확장되지 않아 어떤 명령과도 매칭되지 않는 사문이었다 (fail-closed — 보안 문제는 아니나 사전 승인 효과 없음). 공식 permissions 문서의 와일드카드 시맨틱(`*` 는 공백 포함 임의 문자열 매칭)에 따라 `Bash(python */scripts/render.py *)` 로 교체하여 실제로 매칭되는 최소 패턴으로 정정함.

8. 현행 구조 스냅샷 (2026-07-02 기준)

최초 리뷰(6장까지) 이후 반영된 변경 사항의 요약. 상세 이력은 git log 참조.

하이브리드 파이프라인 (파서 우선)

단계	담당	스크립트	성질
사실 추출 (1-A~1-H)	파서	<code>scripts/extract.py</code>	결정적 — 같은 코드 → 같은 사실 표 (SHA-256 검증)
유저플로우 다이어그램	파서 + LLM	<code>scripts/flowgen.py</code>	스켈레톤은 파서, 라벨·[추정] 점선은 LLM 보강
해석·서술 (Step 2~3)	LLM	—	사실 표 밖 내용 도입 금지 원칙
렌더링	파서	<code>scripts/render.py</code>	PDF+HTML 쌍 기본 (<code>--format pdf,html</code>), DOCX 옵션

현행 `allowed-tools` (두 스킬 공통)

```
Read, Glob, Write, Bash(find *), Bash(ls *),
Bash(pip install weasyprint *), Bash(pip install markdown *), Bash(pip install python-docx *),
Bash(python */scripts/render.py *), Bash(python */scripts/flowgen.py *), Bash(python */scripts/flowgen.py *),
Bash(python3 */scripts/render.py *), Bash(python3 */scripts/flowgen.py *), Bash(python3 */scripts/flowgen.py *),
Bash(pip3 install weasyprint *), Bash(pip3 install markdown *), Bash(pip3 install python-docx *)
```

- `python3` / `pip3` 변형은 macOS(스톡 환경에 `python` 부재) 대응.
- 여전히 임의 Python 실행은 사전 승인되지 않음 — 동봉 스크립트 3종만 허용.

대형 코드베이스 분할 모드

- Step 0에서 소스 30개 초과 판정 시, 모듈 단위로 서브에이전트에 `extract.py` 실행을 위임(격리 컨텍스트, 사실 표만 회수) 후 병합. 공식 `sub-agents/large-codebases` 패턴.

문서 목차 확장

- 에러/메시지 카탈로그(문구 원문 보존), 상태 관리 & 데이터 흐름, 외부 연동 인벤토리, 이벤트/트래킹 명세(KPI 근거 연결), As-Is 스냅샷(커밋 해시 기준선), 추적성 매트릭스.

플랫폼 안내 (README·SKILL.md 4-C)

- Windows PDF: MSYS2 + `pacman -S mingw-w64-ucrt-x86_64-pango` (WeasyPrint 공식 권장), 실패 시 `--format docx,html` 폴백. macOS: `brew install weasyprint`.

공유 사본 관리

- `reference.md` + `scripts/` 3종은 두 스킬에 동일 사본 — `tools/check-sync.sh` 로 검증.

9. 검증 상태 — 무엇이 실제로 증명됐고 무엇이 아직인가 (2026-07-02)

과대평가를 막기 위해, "구현됨"과 "실전 검증됨"을 구분해 명시한다.

항목	증명된 것	아직 증명 안 된 것
<code>extract.py</code> 결정성	mock-shop(6파일)에서 2회 실행 SHA-256 동일	실전 코드베이스에서의 커버리지 — 정규식 기반이라 Next.js 파일 라우팅, Vue <code><script setup></code> , CSS-in-JS, 비표준 상태관리 패턴은 놓칠 수 있음 (<code>reference.md</code> 1-l)

항목	증명된 것	아직 증명 안 된 것
분할 모드(대형 코드베이스)	SKILL.md에 절차로 명문화, 공식 sub-agents 패턴 근거	30개 초과 실제 프로젝트에서 실행된 적 없음 — 모듈 분할 기준, 병합 로직의 실전 판단 미검증
전체 파이프라인 (extract→flowgen→render)	개별 스크립트 3종 각 각 CLI로 직접 실행해 동작 확인	Claude Code가 SKILL.md를 읽고 실제로 이 순서를 트리거하는 end-to-end 실행은 미실행 — 스크립트는 맞아도 스킬 지침이 의도대로 유도 하는지는 별개

권고: 조직 표준 도구로 채택하기 전에, 실제 사내 코드베이스 1~2건에 그대로 실행해 `extract.py`의 추출 통계가 합리적인지(라우트/컴포넌트 수가 코드 규모와 맞는지) 확인할 것. 비정상적으로 낮으면 1-의 파서 커버리지 경고 절차가 트리거되는지도 함께 확인한다.

10. 코드 감사 결과 및 수정 이력 (2026-07-05)

`reverse-spec` / `reverse-prd` (및 자매 스킬 `reverse-backend`)의 실전 검증 과정에서 발견된 코드/구조 문제를 감사하고 수정했다. 모든 수정은 `mock-shop` 2회 실행 SHA-256 동일(결정성 유지) + 전체 렌더 파이프라인(PDF/HTML/DOCX) 재검증을 거쳤다.

#	문제	심각도	수정 내용
1	DOCX 테이블에서 <code>\ </code> 이스케이프 처리가 HTML/PDF와 DOCX 경로에서 서로 달라, 조건문에 <code>\\ \\</code> (JS 논리 OR)가 있으면 DOCX 테이블의 뒤 컬럼(근거 파일 등)이 통째로 사라짐	높음 (데이터 유실)	<code>extract.py</code> 에 <code>_escape_cell()</code> (백틱 값은 이스케이프 생략), <code>render.py</code> 에 백틱-이스케이프 인식 <code>_split_table_row()</code> 도입. HTML의 백슬래시 노출 버그도 함께 해소
2	<code>reverse-backend</code> 에서 발견한 "주석 처리된 죽은 코드를 활성 정책으로 오탐" 버그가 프론트엔드용 <code>extract.py</code> 에는 이식되지 않은 채 남아있었음	높음 (오탐)	<code>_blank_full_line_comments()</code> 추가 (<code>//</code> , 한 줄 HTML 주석 제외) 후 <code>read_sources()</code> 에 적용
3		중간	<code>_find_matching_paren()</code> (괄호 카운팅)으로 두 곳 모두 교체. 부수효과로

#	문제	심각도	수정 내용
	<code>fetch(...)</code> 와 <code>if (...)</code> 정규식이 중첩 괄호(예: <code>.test(email)</code> , <code>JSON.stringify({...})</code>)에서 조기 매칭 종료 — 실제로 mock-shop의 이메일 정규식 검증 규칙이 이 버그로 누락되고 있었음	(실측 정확도)	<code>extract_rules / extract_messages</code> 의 중복 로직을 <code>_find_if_return_pairs()</code> 로 통합(#6 해소)
4	<code>flowgen.py</code> 의 SVG <code>viewBox</code> 높이가 역방향(복귀) 엣지의 우회 경로 좌표와 별개 공식으로 계산돼, 노드가 많으면 엣지가 캔버스 밖으로 잘릴 수 있었음	낮음	엣지 지오메트리를 먼저 계산해 실제 필요 높이를 구한 뒤 <code>viewBox</code> 를 확정하도록 재구성
5	<code>reference.md</code> 가 "민감 정보는 경고로 표시"라고 선언하지만 프론트엔드 <code>extract.py</code> 에는 이를 강제하는 스캔 로직이 없었음 (원칙과 구현의 불일치)	중간	<code>reverse-backend</code> 의 검증된 시크릿 스캔 로직 (<code>extract_secret_findings</code>)을 이식 — <code>reference.md</code> 1-J로 등록, 값은 절대 출력하지 않고 파일 경로·패턴 종류·git 추적 여부만 보고
6	<code>extract_rules / extract_messages</code> 가 동일 정규식을 독립 구현해 한쪽만 고치면 drift 발생	낮음 (구조)	#3 수정 과정에서 <code>_find_if_return_pairs()</code> 로 통합해 해소
7	<code>flowgen.py</code> 에 노드 수 상한이 없어 대형 코드베이스에서 거대한 SVG가 만들어질 수 있었음	낮음	<code>MAX_NODES = 60</code> 상한 추가, 초과 시 잘라내고 <code>stderr</code> 경고

모든 항목은 `reverse-prd/scripts/` 에서 수정 후 `reverse-spec/scripts/` 로 동기화, `tools/check-sync.sh` 통과 확인. 저장소에 커밋된 샘플(mock-shop PRD, flow SVG)은 수정 전후 바이트 단위로 동일함을 확인해 재생성하지 않았다(회귀 없음의 증거).

11. 재점검 결과 — 트레일링 주석 사각지대 발견·수정 (2026-07-07)

10장의 #2(주석 처리된 죽은 코드 오탐 방지) 수정을 재점검하는 과정에서, 그 수정 자체의 사각지대를 추가로 발견해 수정했다.

발견: `_blank_full_line_comments()` 는 한 줄 전체가 `//` 로 시작하는 경우만 다뤘다. 실제로는 코드; `// 주석` 형태의 **트레일링(줄 끝) 주석**이 통째로 주석인 경우보다 훨씬 흔한데, 이 경우 주석 안에 쓰

인 가짜 코드가 여전히 오탐될 수 있었다. 재현:

```
doRealThing(); // fetch("/api/fake-endpoint") 참고용 예시 같은 줄에서 주석 안의
/api/fake-endpoint 가 실제 API 호출처럼 추출됨.
```

1차 수정: 공백 뒤에 오는 `//` 를 트레일링 주석 시작으로 보고 그 지점에서 라인을 잘라내도록 추가 (공백 없는 `https://`, `//cdn.example.com/...` 프로토콜 상대 URL은 보존).

1차 수정의 회귀: mock-shop 회귀 테스트에서 API 추출 수가 2→1로 감소. 원인 추적 결과, `await login(email, password); // POST /api/auth/login` 처럼 **의도적으로 남겨둔 API 문서화 주석**(코드가 헬퍼 함수로 추상화돼 있어 실제 엔드포인트가 주석으로만 남아있는 경우)까지 트레일링 주석으로 오인해 함께 잘라내고 있었다. `extract_api_calls` 의 전용 패턴은 원래 이런 주석 안의 API 표기를 **의도적으로** 찾도록 설계된 것이었다.

최종 수정: 죽은 코드(실제 JS 문법)와 `// GET|POST|PUT|PATCH|DELETE /path` 형태의 의도적 문서화 표기(유효한 JS 문법이 아닌 순수 주석 규칙)를 구분하는 `_API_ANNOTATION` 정규식을 추가해, 후자는 전체 줄/트레일링 주석 제거 대상에서 모두 예외 처리했다.

검증: mock-shop 재실행 결과 가짜 트레일링 API는 여전히 안 잡히고, 의도된 API 문서화 주석 2건(로그인·주문 엔드포인트)은 정상 복원, 2회 실행 해시 동일 (결정성 유지), PDF/HTML/DOCX 파이프라인 재검증 통과.

교훈: 오탐 방지 수정 자체가 새로운 종류의 오탐/누락을 만들 수 있다 — 특히 "주석을 지운다"처럼 광범위한 전처리는 그 주석을 의도적으로 읽는 다른 기능과 충돌할 수 있으므로, 수정 후에도 다른 추출 항목에 회귀가 없는지 전체 통계를 반드시 재확인해야 한다.

출처

- <https://code.claude.com/docs/en/skills.md> (프론트매터 필드 표, 디렉토리 구조, `{CLAUDE_SKILL_DIR}`)
- <https://code.claude.com/docs/en/commands.md> (커맨드↔스킬 통합, `$ARGUMENTS`)
- <https://code.claude.com/docs/en/mcp.md> (MCP 도구 네이밍 `mcp_server_tool`)